

SMART CONTRACT AUDIT REPORT

for

2Factor Finance

Prepared By: Xiaomi Huang

PeckShield

September 9, 2026

Document Properties

Client	Fragments, Inc.
Title	Smart Contract Audit Report
Target	2Factor
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	September 9, 2026	Xuxian Jiang	Final Release
1.0-rc1	September 7, 2026	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1 Introduction 4
- 1.1 About 2Factor Finance . . . 4
- 1.2 About PeckShield . . . 5
- 1.3 Methodology . . . 5
- 1.4 Disclaimer . . . 7

2 Findings 9
- 2.1 Summary . . . 9
- 2.2 Key Findings . . . 10

3 Detailed Results 11
- 3.1 Missing minOpUsdVal Check in maxAcceptableTerms() . . . 11
- 3.2 Inconsistent Rebalance USD Value Calculation in UsdPerpSrVault . . . 13
- 3.3 Improved Precision in Rebalance Perp Amount Calculation . . . 15
- 3.4 Token Allowance Reset-After-Use in Router . . . 16
- 3.5 Trust Issue of Admin Keys . . . 18

4 Conclusion 21

References 22

1 Introduction

Given the opportunity to review the design document and related smart contract source code of the 2Factor Finance protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About 2Factor Finance

2Factor Finance is an innovative protocol for perpetual tranches (perp-tranches). The system creates senior (Sr) and junior (Jr) tranches from underlying assets, wrapped as perpetual ERC20 notes with automatic rollover and rebalancing mechanics. It enables users to gain leveraged or de-leveraged exposure to underlying assets through a two-tranche bond system. Users can mint perpetual senior notes (PerpSr) for lower-volatility exposure or perpetual junior notes (PerpJr) for leveraged exposure, backed by dynamically managed reserves of tranched bonds. The basic information of the audited protocol is as follows:

Table 1.1: Basic Information of The 2Factor Finance Protocol

Item	Description
Client	Fragments, Inc.
Website	https://2factor.finance
Type	EVM Smart Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	September 9, 2026

In the following, we show the Git repository of reviewed files and the commit hash value used in

this audit.

- https://github.com/fragmentsorg/2factor.git (2e527da)

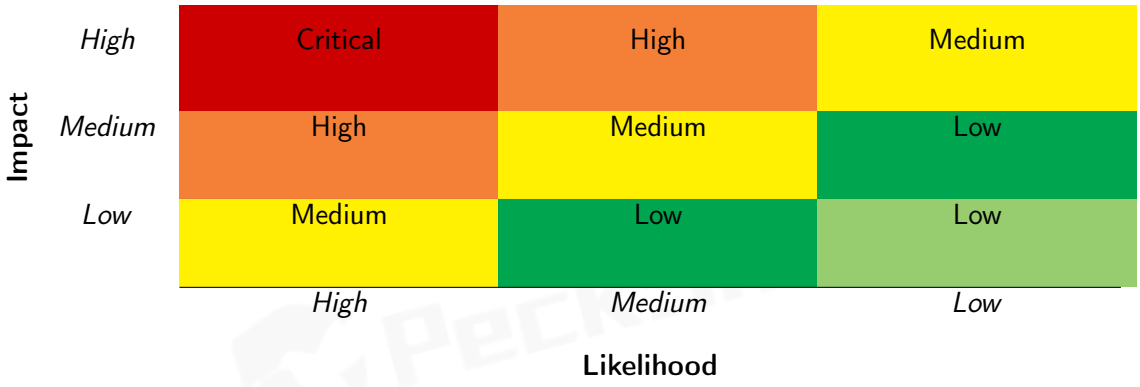
And here is the commit ID after all fixes for the issues found in the audit have been checked in:

- https://github.com/fragmentsorg/2factor.git (2bd78c0)

1.2 About PeckShield

PeckShield Inc. [11] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification



1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [9], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the design and implementation of the 2Factor Finance protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	2	■■
Low	3	■■■
Informational	0	
Total	5	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in Section 3.

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 2 medium-severity vulnerabilities and 3 low-severity vulnerabilities.

Table 2.1: Key 2Factor Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Low	Missing minOpUsdVal Check in maxAcceptableTerms()	Business Logic	Resolved
PVE-002	Low	Inconsistent Rebalance USD Value Calculation in UsdPerpSrVault	Coding Practices	Resolved
PVE-003	Medium	Improved Precision in Rebalance Perp Amount Calculation	Numeric Errors	Resolved
PVE-004	Low	Token Allowance Reset-After-Use in Router	Coding Practices	Resolved
PVE-005	Medium	Trust on Admin Keys	Security Features	Resolved

Besides recommending specific countermeasures to mitigate these issues, we also emphasize that it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms need to kick in at the very moment when the contracts are being deployed in mainnet. Please refer to Section 3 for details.

3 Detailed Results

3.1 Missing minOpUsdVal Check in maxAcceptableTerms())

- ID: PVE-001
- Severity: Low
- Likelihood: Medium
- Impact: Low
- Target: `UsdPerpSrVault`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

The `2Factor Finance` protocol has a core `UsdPerpSrVault` contract that is a `USD`-denominated vault holding `USD` and `perpSr`. To better facilitate the passive rebalance, it provides a view function `maxAcceptableTerms()` to return the largest terms the vault would settle right now in each direction. Our analysis shows it does not take a key risk parameter (`minOpUsdVal`) into account and the key risk parameter is enforced in both active and passive rebalances.

To elaborate, we show below the implementation of this specific function `maxAcceptableTerms()`. As mentioned earlier, it computes the largest terms the vault would settle right now in each direction or the pair named with both amounts 0 for a direction it will not trade. However, current logic does not consider the minimum `USD` value a vault operation must move (defined as `minOpUsdVal`). As a result, current computation result may fail to perform a passive rebalance if the computed rebalance value amount is smaller than `minOpUsdVal`. To fix, we may need to enforce the following statement, i.e., `require(_underlyingToUsdVal(maxRebalanceUnderlying, underlyingPrice)>= minOpUsdVal);`.

```
882   function maxAcceptableTerms()
883     external
884     view
885     returns (SwapTerms memory underlyingIn, SwapTerms memory usdIn)
886   {
887     usdIn = SwapTerms(usd, underlying, 0, 0);
888     underlyingIn = SwapTerms(underlying, usd, 0, 0);
889
890     uint256 maxRebalanceUnderlying;
```

```
891     uint256 underlyingPrice;
892     bool isMintingPerp;
893     (
894       maxRebalanceUnderlying,
895       underlyingPrice,
896       isMintingPerp,
897       ,
898
899     ) = rebalanceCapacity();
900
901     (
902       bool active,
903       bool sessionIsMintingPerp_,
904       uint256 sessionBudgetUsdVal_,
905
906     ) = sessionCapacity();
907
908     // Sized only when the DR gap, the session budget and the direction the open
909     // session locked all allow it; otherwise both legs stay at 0.
910     if (
911       maxRebalanceUnderlying > 0 &&
912       sessionBudgetUsdVal_ > 0 &&
913       (!active || isMintingPerp == sessionIsMintingPerp_)
914     ) {
915       // Both in underlying, the unit `swap` caps its input in.
916       maxRebalanceUnderlying = Math.min(
917         maxRebalanceUnderlying,
918         _usdValToUnderlying(sessionBudgetUsdVal_, underlyingPrice)
919       );
920
921       // The vault takes underlying in exactly when it is accumulating perpSr.
922       if (isMintingPerp) {
923         underlyingIn = _buyUnderlyingTerms(
924           maxRebalanceUnderlying,
925           underlyingPrice
926         );
927       } else {
928         // NOTE: We account for the flash redeem overshoot.
929         maxRebalanceUnderlying = maxRebalanceUnderlying.mulDiv(
930           ONE - maxFlashRedeemFeePerc,
931           ONE
932         );
933         usdIn = _sellUnderlyingTerms(maxRebalanceUnderlying, underlyingPrice);
934       }
935     }
936   }
```

Listing 3.1: `UsdPerpSrVault::maxAcceptableTerms()`

Recommendation Improve the above-mentioned routine to properly enforce the `minOpUsdVal` parameter.

Status This issue has been fixed by this commit: `3807c2d`.

3.2 Inconsistent Rebalance USD Value Calculation in UsdPerpSrVault

- ID: PVE-002
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `UsdPerpSrVault`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

As mentioned in Section 3.1, the `2Factor Finance` protocol has a core `UsdPerpSrVault` contract that is subject to either active or passive rebalance. For each rebalance, it computes the rebalance `USD` value and ensure it is always above the risk parameter `minOpUsdVal`. While reviewing the two rebalance approaches, we notice an inconsistency when computing the rebalance `USD` value.

To elaborate, we show below the implementation of two related functions, i.e., `_rebalanceSellUnderlying()` and `_swapSellUnderlying()`. The former is used in active rebalance to flash-redeem `perpSr` into `underlying`, which is then swapped to `USD`. And the latter is the former's passive mirror with the swapper standing in for the token swap between `underlying` and `USD`. It comes to our attention that the active rebalance computes the rebalance `USD` value with `underlyingAmtRedeemed` (lines 1159–1162) while the passive rebalance computes with `underlyingVal` (line 1235) that is adjusted, not the actual redeemed amount. Note a similar inconsistency is observed in their respective buy counterparts, i.e., `_rebalanceBuyUnderlying()` and `_swapBuy(Underlying()`.

```
1126  function _rebalanceSellUnderlying(
1127    IDexStrategy strategy,
1128    uint256 underlyingAmtAvail,
1129    uint256 underlyingPrice,
1130    IPerpController.SystemReserveState memory state
1131  ) private returns (uint256 rebalanceUsdVal) {
1132    SwapTerms memory s = _sellUnderlyingTerms(
1133      underlyingAmtAvail,
1134      underlyingPrice
1135    );
1136
1137    (uint256 underlyingAmtOut, ) = strategy.maxFillable(
1138      s.tokenOut,
1139      s.tokenIn,
1140      s.maxTokenOutAmt,
1141      s.minTokenInAmt
1142    );
```

```
    uint256 underlyingAmtRedeemed = _flashRedeemPerpSr(
      state.perpSr,
      underlyingAmtOut
    );
    uint256 usdAmtIn = strategy.trade(
      s.tokenOut,
      s.tokenIn,
      underlyingAmtRedeemed
    );
    _checkRateFloor(
      underlyingAmtRedeemed,
      usdAmtIn,
      s.maxTokenOutAmt,
      s.minTokenInAmt
    );

    rebalanceUsdVal = _underlyingToUsdVal(
      underlyingAmtRedeemed,
      underlyingPrice
    );
  }
```

Listing 3.2: `UsdPerpSrVault::_rebalanceSellUnderlying()`

```
  function _swapSellUnderlying(
    uint256 usdAmtIn,
    uint256 underlyingAmtOut,
    uint256 underlyingPrice,
    uint256 maxRebalanceUnderlying,
    IPerpController.SystemReserveState memory state
  ) private returns (uint256 rebalanceUsdVal) {
    // We adjust the redemption value to account for flash fees.
    uint256 underlyingVal = underlyingAmtOut.mulDiv(
      ONE,
      ONE - maxFlashRedeemFeePerc,
      Math.Rounding.Ceil
    );
    if (underlyingVal > maxRebalanceUnderlying) {
      revert OpUnavailable('capacity');
    }
    rebalanceUsdVal = _underlyingToUsdVal(underlyingVal, underlyingPrice);

    usd.safeTransferFrom(msg.sender, address(this), usdAmtIn);

    _flashRedeemPerpSr(state.perpSr, underlyingVal);

    underlying.safeTransfer(msg.sender, underlyingAmtOut);
  }
```

Listing 3.3: `UsdPerpSrVault::_swapSellUnderlying()`

Recommendation Revise the above-mentioned routines to ensure the rebalance `USD` values are

consistently computed.

Status This issue has been fixed by this commit: `a859dcb`.

3.3 Improved Precision in Rebalance Perp Amount Calculation

- ID: PVE-003
- Severity: Medium
- Likelihood: Medium
- Impact: Low
- Target: `UsdPerpSrVault`
- Category: Numeric Errors [8]
- CWE subcategory: CWE-190 [2]

Description

`SafeMath` is a widely-used Solidity `math` library that is designed to support safe `math` operations by preventing common overflow or underflow issues when working with `uint256` operands. While it indeed blocks common overflow or underflow issues, the lack of `float` support in `Solidity` may introduce another subtle, but troublesome issue: precision loss. In this section, we examine one possible precision loss source that stems from the arithmetic ceiling/floor operations.

To elaborate, we show below the implementation of the affected `flashRedeemPerpWorth()` library function that is designed to flash-redeem `perp` for a given worth `underlyingVal` (at the pre-trade reserve rate) and return the post-fee underlying delivered. It has a rather straightforward logic in calculating the respective `perp` amount for the given worth `underlyingVal` (lines 270−274) and then flash-redeem the `perp` amount. It comes to our attention that the given `underlyingVal` is the maximum underlying amount for rebalance, which indicates the need of applying the `Math.Rounding.Floor` operation, instead of current `Math.Rounding.Ceil` (line 273).

```
264   function flashRedeemPerpWorth(
265     IPerpController controller,
266     IPerp.ReserveState memory perpState,
267     uint256 underlyingVal,
268     bool isPerpSrVault
269   ) internal returns (uint256 underlyingDelivered, uint256 feePerc) {
270     uint256 perpAmt = underlyingToPerp(
271       perpState,
272       underlyingVal,
273       Math.Rounding.Ceil
274     );
275     (underlyingDelivered, feePerc) = flashRedeemPerp(
276       controller,
277       perpAmt,
278       isPerpSrVault
279     );
```

```
280   }
```

Listing 3.4: PerpVaultHelpers::flashRedeemPerpWorth()

Recommendation Revise the above calculation to apply the intended floor operation to better mitigate possible precision loss.

Status This issue has been fixed by this commit: e7c0c8a.

3.4 Token Allowance Reset-After-Use in Router

- ID: PVE-004
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: Router
- Category: Coding Practices [6]
- CWE subcategory: CWE-1126 [1]

Description

The core contracts in 2Factor include a convenience Router contract that is designed to batch multi-step perp operations into single transactions. Specifically, it provides a number of user-facing functions (e.g., trancheAndMint(), redeem3(), mint2(), redeem2(), flashMint2(), and flashRedeem2()) to facilitate the interaction with the PerpController. In the analysis of these user-facing functions, we notice the standing allowance of related tokens to external contracts that may not be trusted.

In the following, we use the trancheAndMint() function as an example. By design, it allows a user to deposit underlying into the latest bond, receive senior/junior tranches, and mint one or both perps in a single transaction. However, it comes to our attention that this function approves a number of external contracts for token spending, including address(bondInfo.manager) (line 103) and address(controller) (lines 113 and 120). And these external contracts are all provided by the calling user and hence considered not trustworthy. With that, there is a need to reset the standing allowance after the use.

```
71  function trancheAndMint(
72    IPerpController controller,
73    uint256 underlyingAmtIn,
74    uint256 minPerpSrAmtOut,
75    uint256 minPerpJrAmtOut,
76    bool mintPerpSr,
77    bool mintPerpJr,
78    bool doUpkeep,
79    DRSlippage memory drSlippage
80  )
81    external
```

```
82      validateDRRange(controller, drSlippage)
83      returns (
84        uint256 perpSrAmtOut,
85        uint256 perpJrAmtOut,
86        TokenAmount memory returnedSrTr,
87        TokenAmount memory returnedJrTr
88      )
89    {
90      IERC20 underlying = controller.underlying();
91
92      // 1. Pull underlying from caller
93      underlying.safeTransferFrom(msg.sender, address(this), underlyingAmtIn);
94
95      // 2. Optionally flush pending bond issuance so latestBondInfo reflects
96      //    the current window.
97      if (doUpkeep) {
98        controller.upkeep();
99      }
100
101     // 3. Deposit underlying into the latest bond -> receive sr + jr tranches
102     IBondIssuer.BondInfo memory bondInfo = controller.latestBondInfo();
103     _ensureApproval(underlying, address(bondInfo.manager), underlyingAmtIn);
104     (returnedSrTr, returnedJrTr) = bondInfo.manager.deposit(
105       bondInfo.bondID,
106       underlyingAmtIn
107     );
108
109     // 4. Approve and mint perps for the selected sides
110     if (mintPerpSr) {
111       _ensureApproval(
112         returnedSrTr.token,
113         address(controller),
114         returnedSrTr.amount
115       );
116     }
117     if (mintPerpJr) {
118       _ensureApproval(
119         returnedJrTr.token,
120         address(controller),
121         returnedJrTr.amount
122       );
123     }
124
125     (perpSrAmtOut, perpJrAmtOut, , ) = controller.mint2(
126       TokenAmount(returnedSrTr.token, mintPerpSr ? returnedSrTr.amount : 0),
127       TokenAmount(returnedJrTr.token, mintPerpJr ? returnedJrTr.amount : 0),
128       minPerpSrAmtOut,
129       minPerpJrAmtOut
130     );
131
132     // 5. Return unused tranches to caller
133     if (mintPerpSr) {
```

```
134       returnedSrTr.amount = 0;
135     } else {
136       _forwardERC20(returnedSrTr.token, returnedSrTr.amount);
137     }
138     if (mintPerpJr) {
139       returnedJrTr.amount = 0;
140     } else {
141       _forwardERC20(returnedJrTr.token, returnedJrTr.amount);
142     }

144     // 6. Transfer minted perps to caller
145     _forwardERC20(controller.perpSr(), perpSrAmtOut);
146     _forwardERC20(controller.perpJr(), perpJrAmtOut);
147   }
```

Listing 3.5: `Router::trancheAndMint()`

Recommendation Reset standing allowance, if any, on external contracts. Or we may whitelist the `PerpController` contract to ensure only trusted contracts may be allowed to have standing allowance. Note this issue also affect other functions, including `mint2()` and `flashMint2()`.

Status This issue has been fixed by this commit: `23be335`.

3.5 Trust Issue of Admin Keys

- ID: PVE-005
- Severity: Medium
- Likelihood: Low
- Impact: Medium
- Target: `Multiple Contracts`
- Category: Security Features [5]
- CWE subcategory: CWE-287 [3]

Description

In the `2Factor Finance` protocol, there is a special administrative account, i.e., `owner`. This `owner` account plays a critical role in governing and regulating the system-wide operations (e.g., configure various settings, assign keepers, and execute privileged operations). It also has the privilege to control or govern the flow of assets within the protocol contracts. In the following, we examine the privileged account and the related privileged accesses in current contracts.

```
376   function updateKeeper(address keeper_) public onlyOwner {
377     keeper = keeper_;}

379   function updateSwapper(address swapper, bool allowed) public onlyOwner {
380     isSwapper[swapper] = allowed;}

382   function updateTargetDR(
```

```
383       uint96 targetDR_,
384       DRRange memory eqDRRange_) public onlyOwner {
385       if (targetDR_ <= 0) revert InvalidParamDefn();
386       if (eqDRRange_.lower > targetDR_ || eqDRRange_.upper < targetDR_) {
387         revert InvalidRange();
388       }
389       targetDR = targetDR_;
390       eqDRRange = eqDRRange_;}

392     function updateOracleDeviationPerc(int64 perc) public onlyOwner {
393       if (perc <= -int256(ONE) || perc >= int256(ONE)) {
394         revert InvalidPerc();
395       }
396       oracleDeviationPerc = perc;}

398     function updateDexStrategy(
399       IDexStrategy newStrategy) public onlyOwner whenStrategyHealthy(newStrategy) {
400       IDexStrategy old = dexStrategy;
401       if (address(old) != address(0)) {
402         usd.forceApprove(address(old), 0);
403         underlying.forceApprove(address(old), 0);
404       }

406       dexStrategy = newStrategy;
407       usd.forceApprove(address(newStrategy), type(uint256).max);
408       underlying.forceApprove(address(newStrategy), type(uint256).max);}

410     function updateOpLimits(
411       uint128 minOpUsdVal_,
412       uint64 maxFlashMintFeePerc_,
413       uint64 maxFlashRedeemFeePerc_) public onlyOwner {
414       if (maxFlashMintFeePerc_ >= ONE || maxFlashRedeemFeePerc_ >= ONE) {
415         revert InvalidPerc();
416       }
417       minOpUsdVal = minOpUsdVal_;
418       maxFlashMintFeePerc = maxFlashMintFeePerc_;
419       maxFlashRedeemFeePerc = maxFlashRedeemFeePerc_;}

421     function updateSessionParams(
422       uint32 freqSec_,
423       uint32 offsetSec_,
424       uint32 activeRebalanceDelaySec_,
425       uint64 maxDeploymentVelocityPerc_) public onlyOwner {...}

427     function updateHealthCheckParams(
428       IPriceOracle usdOracle_,
429       uint64 maxUsdDeviationPerc_,
430       uint64 maxUnderlyingDeviationPerc_) public onlyOwner {...}

432     function pause() external onlyKeeper {
433       _pause();}
```

```
435   function unpause() external onlyKeeper {
436     _unpause();}
```

Listing 3.6: Example Privileged Operations in `UsdPerpSrVault`

We understand the need of the privileged functions for proper contract operations, but at the same time the extra power to these privileged accounts may also be a counter-party risk to the contract users. Therefore, we list this concern as an issue here from the audit perspective and highly recommend making these privileges explicit or raising necessary awareness among protocol users.

In the meantime, the audited protocol makes use of the proxy contracts to allow for future upgrades. The upgrade is a privileged operation and the management of the related admin key also falls in this trust issue.

Recommendation Promptly transfer the privileged account to the intended `DAO`-like governance contract. All changes to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

Status The issue has been resolved as the team clarifies the use of a publicly documented multisig address as the `owner` account.

PeckShield

4 Conclusion

In this audit, we have analyzed the design and implementation of the 2Factor Finance protocol for perpetual tranches. The protocol creates senior and junior tranches from underlying assets, wrapped as perpetual ERC20 notes with automatic rollover and rebalancing mechanics. It enables users to gain leveraged or de-leveraged exposure to underlying assets through a two-tranche bond system. Users can mint perpetual senior notes for lower-volatility exposure or perpetual junior notes for leveraged exposure, backed by dynamically managed reserves of tranched bonds. The current code base is clearly organized and those identified issues are promptly confirmed and fixed.

Meanwhile, we need to emphasize that Solidity-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

PeckShield

References

[1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. https://cwe.mitre.org/data/definitions/1126.html.

[2] MITRE. CWE-190: Integer Overflow or Wraparound. https://cwe.mitre.org/data/definitions/190.html.

[3] MITRE. CWE-287: Improper Authentication. https://cwe.mitre.org/data/definitions/287.html.

[4] MITRE. CWE-770: Allocation of Resources Without Limits or Throttling. https://cwe.mitre.org/data/definitions/770.html.

[5] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[6] MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

[7] MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

[8] MITRE. CWE CATEGORY: Numeric Errors. https://cwe.mitre.org/data/definitions/189.html.

[9] MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

[10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[11] PeckShield. PeckShield Inc. https://www.peckshield.com.

PeckShield